

# UNG 组内 PG 构建 GPU 并行优化规格文档

## (Spec Coding 工作流版)

### 一、宪章 (Constitution)

此为最高准则，任何实现、优化或重构都不得违背。

#### • 代码质量与可维护性:

- **模块化与封装:** GPU 加速代码必须封装在独立的、有清晰接口的模块/类中（如 `GpuPgBuilder`），与现有 CPU 逻辑（`uni_nav_graph.cpp`）低耦合。禁止将 CUDA `__global__` 函数原型直接散布在 C++ 头文件中。
- **代码复用:** 优先使用 CUB、Thrust 等成熟的 GPU 并行库 primitives，避免重复造轮子。所有手写的核函数（Kernel）必须附带其设计意图和线程/块映射策略的注释。
- **命名与风格:** 遵循现有项目的命名约定（如 `_camelCase` 成员变量）。GPU 相关代码（文件、函数、变量）应有明确前缀或命名空间（如 `gpu::`）以示区分。

#### • 性能目标:

- **端到端加速比:** 针对 `build_graph_for_all_groups` 函数，在典型数据集（如 SIFT1M）上，GPU 版本应比优化的多线程 CPU 版本（`omp parallel for`）快至少 **5x**。
- **GPU 利用率:** 在处理大型组（向量数 > 100,000）时，`nvidia-smi` 显示的 GPU-Util 应持续高于 **80%**。
- **延迟敏感性:** 整个构图流程（`build_graph_for_all_groups`）的总耗时应与组的数量和大小呈亚线性关系，避免少量大组阻塞整体流程。

#### • 显存预算 (Memory Budget):

- **上限约束:** 任何单张 GPU 的显存占用峰值不得超过 **16 GB**，以保证在主流专业级显卡上的可运行性。
- **无动态显存分配:** 在 GPU Kernel 执行期间，严禁使用 `new`、`malloc` 或任何形式的动态显存分配。所有必需的缓冲区（邻居池、候选集、距离矩阵等）必须在 Host 端预先计算大小并一次性分配。
- **内存复用:** 积极复用临时显存。例如，距离计算缓冲区可在不同组的构建任务之间共享。

#### • 安全与鲁棒性:

- **边界检查:** 所有核函数必须能正确处理空组（0个向量）、小组（向量数 < k）和大数据尺寸（向量数 > 4M）的边界情况而不崩溃。

- **错误处理:** 所有 CUDA API 调用 (如 `cudaMalloc`, `cudaMemcpyAsync`, `kernel_launch`) 必须被宏包裹, 以便在发生错误时能立即捕获并抛出详细的 C++ 异常。
- **数据一致性:** 并发更新邻居列表时, 必须采用无锁 (如原子操作 `atomicCAS`) 或半无锁 (如分块并行、各自写回) 策略, 确保数据无竞争、无撕裂。
- **测试与可验证性:**
  - **可复现性:** 给定相同的输入数据和参数, 两次连续的 GPU 构图运行应产生完全相同的图结构 (邻接表)。随机性来源 (如初始邻居选择) 必须使用固定的种子。
  - **单元测试:** 每个独立的 GPU 模块 (如近邻迭代、剪枝) 都必须有对应的单元测试, 可在隔离环境中验证其正确性。
  - **端到端验证:** 最终生成的图必须通过独立的脚本进行质量评估 (如 Recall@k), 其结果应与 CPU Vamana 版本在同一水平 (下降不超过 1%)。

### 三、技术计划 (Plan)

本节描绘了从架构到实现的蓝图, 编码代理需以此为指导进行模块设计。

#### 3.1 架构设计与 GPU 并行方案

##### 3.1.1 核心思想: 两阶段图构建的 GPU 实现

1. **阶段 A (迭代 kNN 构建):** 此阶段的目标是在 GPU 上高效模拟 NN-Descent 过程, 产出一张高质量的近似 kNN 图。
  - **线程映射:**
    - 每个 **Warp (32 线程)** 负责处理 **一个点** 的邻居更新任务。这种 **Vertex-centric** 的 warp-level 协同能最大限度地利用 warp 内置的高效原语 (如 `__shfl_sync`、`__ballot_sync`), 避免昂贵的全局同步。
  - **访存合并:**
    - 将向量数据以 SoA (Structure of Arrays) 格式存储, 确保一个 warp 内的线程访问连续的内存地址, 触发访存合并。
    - 候选邻居的向量数据也应组织成适合合并访问的批次。
  - **共享内存/寄存器:**
    - 一个点的邻居池 (`L` 个邻居 ID 和距离) 可以加载到 **共享内存 (Shared Memory)** 中, 供 warp 内所有线程高速访问。
    - 每个线程各自计算的一小部分距离结果可暂存于 **寄存器 (Registers)** 中。
  - **多 Stream Pipeline:**
    - 利用多个 CUDA Stream 实现数据拷贝 (H2D)、核函数执行 (Kernel)、结果写回 (D2H) 的流水线化, 隐藏数据传输延迟。

- 例如：Stream 1 计算组  $i$ ，同时 Stream 2 负责将组  $i+1$  的数据从主机拷贝到设备。

## 2. 阶段 B (ANN 剪枝): 此阶段接收阶段 A 的输出，通过并行化的 RNG/NSG/SSG 判据进行剪枝。

### ○ 剪枝策略:

- **RNG (Relative Neighborhood Graph)** 是首选基准。其判据为：对于点  $u$  的两个邻居  $v$  和  $w$ ，如果  $\text{dist}(v, w) < \text{dist}(u, v)$  且  $\text{dist}(v, w) < \text{dist}(u, w)$ ，则  $u$  到  $v$  和  $u$  到  $w$  中较长的一条边可以被剪掉。
- **并行化实现**: 为每个点  $u$  启动一个 CUDA block。Block 内的线程并行检查  $u$  的所有邻居对  $(v, w)$  是否满足 RNG 条件，并使用原子操作在一个标记数组中标记待删除的边。最后，一个单独的 kernel 负责根据标记数组压缩邻接表。

- **度控制**: 剪枝后，若某点的出度仍大于  $\text{max\_degree}$ ，则简单截断，只保留最近的  $\text{max\_degree}$  个邻居。

## 3.2 组间并行与负载均衡

CPU 主机端的调度器 ( $\text{GpuScheduler}$ ) 负责该部分逻辑。

### 1. 代价估计 (Cost Estimation):

- 对于每个组  $g$ ，其构图的计算成本可近似估计为  $\text{Cost}(g) = \text{Size}(g)^{1.2} * \text{Dim}$ ，其中  $\text{Size}(g)$  是组内向量数， $\text{Dim}$  是维度。1.2 是经验系数，反映了迭代算法的非线性复杂度。

### 2. 任务划分策略:

#### ○ 大组分块 (Tiling):

- **When**  $\text{Cost}(g)$  超过一个阈值  $T\_large$  (例如，预计需要超过 200ms 的 GPU 计算时间),
- **Then** 将该组的  $N$  个点在逻辑上切分为多个 **Tiles (块)**，每个 Tile 大小为  $T\_size$  (如 4096 个点)。
- 构图将按 Tile 进行，一个 Tile 内的点只与同 Tile 的其他点以及少量随机采样的“跨 Tile”点进行邻居交换。这是一种分而治之的策略，将大任务分解为可在单个 CUDA Kernel 中高效执行的小任务。

#### ○ 小组合并 (Batching):

- **When** 多个组的  $\text{Cost}$  之和小于一个阈值  $T\_small$ ,
- **Then** 将这些小组的向量数据在显存中拼接起来，作为一个**逻辑大组 (Mega-Group)**，一次性启动一个大的 Kernel 来处理。这避免了为每个小组都启动一次 Kernel 的开销，显著提升了对小任务的处理效率。

### 3. 动态调度与工作窃取 (Work-Stealing):

- 所有待处理的任务 (Tiles 或 Batches) 被放入一个全局的、线程安全的**任务队列**中。

- 每个 CUDA Stream 作为一个工作线程，完成当前任务后，从队列中获取下一个任务执行。
- **If** 队列为空，而已有任务（特别是被 Tiling 的大组）仍在执行中且其内部子任务队列不为空，
- **Then** 空闲的 Stream 可以从繁忙的 Stream 的子任务队列中“窃取”一个 Tile 进行处理。

#### 4. 多 GPU 策略:

- 调度器检测可用的 GPU 数量 `N_gpu`。
- 任务队列中的任务被平均分配给 `N_gpu` 张卡。每张卡独立维护自己的 Stream 和任务执行。
- **结果归并**: 每张卡完成计算后，将生成的邻接表片段异步拷贝回主机的暂存区 (Pinned Memory)，最后由 CPU 线程将所有片段拼接成最终的全局 `_graph`。

### 3.3 与现有 CPU 代码的接口衔接

在 `build_graph_for_all_groups` 函数内部，实现以下衔接逻辑：

#### 1. 创建调度器:

代码块

```
1  // 伪代码
2  void ANNS::UniNavGraph::build_graph_for_all_groups() {
3      // ... 原有omp parallel for之前的代码保持不变 ...
4
5      // 1. 创建并配置GPU调度器
6      GpuSchedulerConfig config;
7      config.num_gpus = detect_gpus();
8      config.stream_per_gpu = 4;
9      config.tiling_threshold = 100000; // 超过10万个点就分块
10
11     GpuScheduler scheduler(config, _base_storage, _group_id_to_range);
12
13     // 2. 准备所有构图任务
14     scheduler.prepare_tasks();
15
16     // 3. 阻塞式执行所有任务
17     scheduler.execute_all();
18
19     // 4. 从调度器获取结果并更新成员变量
20     scheduler.copy_results_to(_graph, _group_entry_points);
21
22     // ... 更新构建时间等统计信息 ...
23 }
24
```

#### 2. 数据流:

- `GpuScheduler` 在构造时接收指向 `_base_storage` 和 `_group_id_to_range` 的指针/引用，以便读取原始数据。
- 调度器在内部管理所有 GPU 内存的分配和释放。
- 执行完毕后，提供一个 `copy_results_to` 方法，将 GPU 上的最终图结构安全地写回 CPU 端的 `_graph` 邻接表中。
- 这种设计确保了 `UniNavGraph` 类不直接依赖 CUDA API，所有 GPU 操作被隔离在 `GpuScheduler` 和相关模块中。

## 四、任务 (Tasks)

此为可执行、可验证的最小工作单元清单。编码代理应逐项实现，并可按并行标记 (P) 并行开发。

 **任务设计原则:** 每个任务都应是单一职责的，具有明确的输入、输出和验收标准，便于独立测试和集成。

| ID                      | 任务目标<br>(Objective)                           | 约束与依赖<br>(Constraints & Dependencies)                                                                          | 产出物路径<br>(Artifact Path)               | 验收测试<br>(Acceptance Test)                                                                            |
|-------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------|------------------------------------------------------------------------------------------------------|
| 阶段 1: GPU 基础设施与数据管理     |                                               |                                                                                                                |                                        |                                                                                                      |
| T01                     | (P) 设计并实现 GPU 内存管理类 <code>DeviceVector</code> | 提供类似 <code>std::vector</code> 的接口，封装 <code>cudaMalloc</code> / <code>cudaFree</code> / <code>cudaMemcpy</code> | <code>gpu/utils/device_vector.h</code> | <code>ASSERT(vec.size() == N);</code><br><code>ASSERT(vec.capacity() &gt;= N);</code><br>能成功分配和释放内存。 |
| T02                     | (P) 实现 CUDA API 错误检查宏 <code>CUDA_CHECK</code> | 封装 CUDA 调用，失败时抛出带文件名和行号的 C++ 异常                                                                                | <code>gpu/utils/cuda_utils.h</code>    | <code>CUDA_CHECK(cudaMalloc(nullptr, 1));</code> 应能捕获并抛出 <code>invalid argument</code> 异常。           |
| T03                     | (P) 实现计时工具 <code>GpuTimer</code>              | 使用 <code>cudaEvent</code> 精确测量 Kernel 执行时间                                                                     | <code>gpu/utils/gpu_timer.h</code>     | 对一个 <code>cudaDeviceSynchronize()</code> 的计时应接近 0ms。                                                 |
| 阶段 2: 组内 PG 构建核心 Kernel |                                               |                                                                                                                |                                        |                                                                                                      |
| T04                     | 实现 <code>kernel_initial</code>                | 为每个点随机选择 <code>L</code> 个初始邻居                                                                                  | <code>gpu/kernels.cuh</code>           | 启动后， <code>d_neighbor_po</code>                                                                      |

|                 |                                                                           |                                                                                                                       |                                 |                                                         |
|-----------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------|---------------------------------------------------------|
|                 | <code>ize_pools</code>                                                    |                                                                                                                       |                                 | <code>ol</code> 中每个点的邻居 ID 均不相同且在合法范围内。                 |
| T05             | (P) 实现 L2/IP 距离计算的 GPU 版本                                                 | 为 SoA 布局优化，利用 <code>__ldg</code> 或 <code>shared memory</code> 提升访存效率                                                  | <code>gpu/distance.cuh</code>   | GPU 计算结果与 CPU 版本相比，误差小于 <code>1e-5</code> 。             |
| T06             | 实现 <code>kernel_generate_candidates</code>                                | (依赖 T04) 从当前邻居的邻居中生成新候选，并进行去重                                                                                         | <code>gpu/kernels.cuh</code>    | 输出的候选池中，任意点的候选列表内无重复 ID。                                |
| T07             | 实现 <code>kernel_update_neighbors</code>                                   | (依赖 T05, T06) 计算与候选的距离，并使用 <code>atomicCAS</code> 更新邻居池                                                               | <code>gpu/kernels.cuh</code>    | 运行后，邻居池中的邻居比运行前更优（平均距离更小）。                              |
| T08             | 将 T06 和 T07 集成到 <code>kernel_nn_descendent_iteration</code> 中             | (依赖 T06, T07) 形成完整的单轮迭代 Kernel                                                                                        | <code>gpu/kernels.cuh</code>    | 单次调用能正确完成一轮邻居发现和更新。                                     |
| T09             | 实现 <code>kernel_prune_graph_rng</code>                                    | (依赖 T05) 根据 RNG 规则，并行地标记待剪枝的边                                                                                         | <code>gpu/kernels.cuh</code>    | 对于一个已知的简单图结构，能正确标记所有不满足 RNG 条件的边。                       |
| T10             | 实现 <code>kernel_compact_graph</code>                                      | (依赖 T09) 根据标记数组，压缩邻接表，确保出度不超过 <code>max_degree</code>                                                                 | <code>gpu/kernels.cuh</code>    | 输入一个带标记的图，输出的图中，每个点的出度 $\leq$ <code>max_degree</code> 。 |
| 阶段 3: 主机端逻辑与调度器 |                                                                           |                                                                                                                       |                                 |                                                         |
| T11             | (P) 定义配置结构体 <code>GpuBuildConfig</code> 和 <code>GpuSchedulerConfig</code> | 包含所有可调参数，如 <code>k</code> , <code>L</code> , <code>alpha</code> , <code>num_gpus</code> , <code>stream_per_gpu</code> | <code>gpu/config.h</code>       | 所有参数均有合理的默认值。                                           |
| T12             | 实现主机端封装类 <code>GpuPgBuilder</code>                                        | (依赖 T08, T10, T11) 封装单组构图                                                                                             | <code>gpu/pg_builder.h</code> , | 调用 <code>build()</code> 方法能成功在 GPU 上                    |



|             |                                                                             |                                                      |                                                               |                                                                                   |
|-------------|-----------------------------------------------------------------------------|------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------|
|             |                                                                             | 的完整流程（迭代+剪枝）                                         | <code>gpu/pg_builder.cu</code>                                | 为一个小规模（如10k点）组构建图。                                                                |
| T13         | 实现代价估计与任务划分逻辑                                                               | 实现 Tiling 和 Batching 策略                              | <code>gpu/scheduler.cpp</code>                                | 输入一组大小各异的 group，能正确输出一个包含 TiledTask 和 BatchedTask 的任务列表。                          |
| T14         | 实现 <code>GpuScheduler</code> 核心调度逻辑                                         | (依赖 T12, T13) 管理 CUDA Stream，实现任务分发和 Work-Stealing   | <code>gpu/scheduler.h</code> , <code>gpu/scheduler.cpp</code> | 在模拟环境下，空闲 stream 能从繁忙 stream 窃取任务。                                                |
| T15         | 实现 <code>GpuScheduler</code> 的结果归并逻辑                                        | 将多 GPU/多任务的图片段安全拷贝回主机内存                              | <code>gpu/scheduler.cpp</code>                                | <code>copy_results_to()</code> 能将 GPU 上的图数据正确、无误地写回 CPU 的 <code>_graph</code> 对象。 |
| 阶段 4: 集成与测试 |                                                                             |                                                      |                                                               |                                                                                   |
| T16         | (P) 编写 <code>GpuPgBuilder</code> 的单元测试                                      | (依赖 T12) 测试单组图的正确性和性能                                | <code>tests/test_gpu_pg_builder.cpp</code>                    | 在 SIFT1M 的一个切片上，召回率达到 0.98。                                                       |
| T17         | (P) 编写 <code>GpuScheduler</code> 的单元测试                                      | (依赖 T14) 重点测试 Tiling, Batching, Work-Stealing 的调度正确性 | <code>tests/test_gpu_scheduler.cpp</code>                     | 对于混合大小的组，调度器能有效利用所有 stream，无死锁。                                                   |
| T18         | 在 <code>build_graph_for_all_groups</code> 中替换为 <code>GpuScheduler</code> 调用 | (依赖 T15) 移除 OpenMP 循环，集成新的 GPU 路径                    | <code>src/uni_nav_graph.cpp</code>                            | 整个 UNG 构图流程能完整跑通，并生成最终图文件。                                                        |
| T19         | 编写端到端 Recall@k 评测脚本                                                         | 可加载 CPU 和 GPU 构建的图，并对其进行搜索质量评测                       | <code>eval/eval_recall.py</code>                              | 脚本能正确加载图、运行搜索并计算召回率。                                                              |
| T20         |                                                                             |                                                      |                                                               | <code>Pass@1</code> 达成。                                                           |

|                |                                                         |                          |
|----------------|---------------------------------------------------------|--------------------------|
| 完成最终的性能和质量对比测试 | (依赖 T18, T19) 对比 GPU 版本与原始 CPU 版本的耗时和 Recall，验证是否满足验收标准 | results/benchm<br>ark.md |
|----------------|---------------------------------------------------------|--------------------------|

## 五、实施 (Implement)

本节提供端到端的执行流程伪代码与测试策略，作为编码代理实现逻辑的参考。

### 5.1 端到端执行顺序

以下伪代码描述了从主机端视角看，构建单个组（或 Tile/Batch）的完整 GPU 任务流。

代码块

```
1 // 伪代码：主机端控制逻辑 (GpuPgBuilder::build)
2
3 function build_single_pg_on_gpu(task, stream):
4     // 0. 获取任务信息
5     vectors = task.get_vectors()
6     num_points = task.num_points
7     dim = task.dim
8
9     // 1. 准备GPU内存
10    d_vectors = allocate_gpu_mem(num_points * dim)
11    d_neighbor_pool = allocate_gpu_mem(num_points * L) // L 是邻居池大小
12    d_candidate_pool = allocate_gpu_mem(num_points * C) // C 是候选池大小
13    d_final_graph = allocate_gpu_mem(num_points * max_degree)
14
15    // 2. 异步数据传输
16    copy_vectors_to_gpu_async(d_vectors, vectors, stream)
17
18    // 3. 阶段 A: 迭代 kNN 构建
19    // 3.1 初始化随机邻居
20    kernel_initialize_pools<<<...>>>(d_neighbor_pool, num_points, L, stream)
21
22    // 3.2 迭代循环
23    for i in 1..MAX_ITERATIONS:
24        // 生成候选 (邻居的邻居)
25        kernel_generate_candidates<<<...>>>(d_neighbor_pool, d_candidate_pool,
stream)
26
27        // 更新邻居池 (包含距离计算和原子更新)
28        kernel_update_neighbors<<<...>>>(d_vectors, d_neighbor_pool,
d_candidate_pool, stream)
29
```



```

30         // [可选] 检查收敛 (可在GPU上完成, 或定期拷贝少量统计数据回CPU检查)
31         if has_converged(stream):
32             break
33
34         // 4. 阶段 B: 剪枝
35         // 4.1 运行RNG剪枝Kernel
36         kernel_prune_graph_rng<<<...>>>(d_vectors, d_neighbor_pool, d_final_graph,
max_degree, stream)
37
38         // 4.2 [可选] 如果剪枝逻辑复杂, 可能需要一个额外的kernel来整理图
39         kernel_compact_graph<<<...>>>(..., stream)
40
41         // 5. 将结果异步写回主机暂存区
42         copy_graph_to_host_async(host_pinned_buffer, d_final_graph, stream)
43
44         // 6. 释放GPU临时内存
45         free_gpu_mem(d_vectors, d_neighbor_pool, d_candidate_pool)
46

```

## 5.2 单元与系统级测试

### • 单元测试 (Unit Testing):

- **方法:** 对每个独立的 GPU Kernel 或主机端模块, 创建专门的测试函数。
- **测试数据:** 使用小规模、人工构造的数据。例如, 测试 `kernel_prune_graph_rng` 时, 可以手动构造一个 5 个点的完全图, 其坐标已知, 因此可以精确地预测哪些边应该被剪枝。
- **断言:**
  - 从 GPU 下载结果到 CPU。
  - 使用 `ASSERT_EQ` 或 `ASSERT_NEAR` 逐点、逐边地验证结果是否与预期完全一致。

### ◦ 示例 (GTest 风格):

#### 代码块

```

1  TEST(GpuKernelsTest, PruneRNG_SimpleCase) {
2      // 1. 在CPU上构造一个5个点的简单图和坐标
3      std::vector<float> vectors = {...};
4      std::vector<int> knn_graph = {...}; // 阶段A的输出
5      std::vector<int> expected_ann_graph = {...}; // 理论剪枝结果
6
7      // 2. 将数据上传到GPU
8      DeviceVector<float> d_vectors(vectors);
9      DeviceVector<int> d_knn_graph(knn_graph);
10     DeviceVector<int> d_final_graph(5 * max_degree);
11

```

```

12 // 3. 调用被测Kernel
13 kernel_prune_graph_rng<<<...>>>(...);
14 cudaDeviceSynchronize();
15
16 // 4. 将结果下载回CPU
17 std::vector<int> actual_ann_graph = d_final_graph.to_cpu();
18
19 // 5. 验证结果
20 ASSERT_EQ(actual_ann_graph, expected_ann_graph);
21 }
22

```

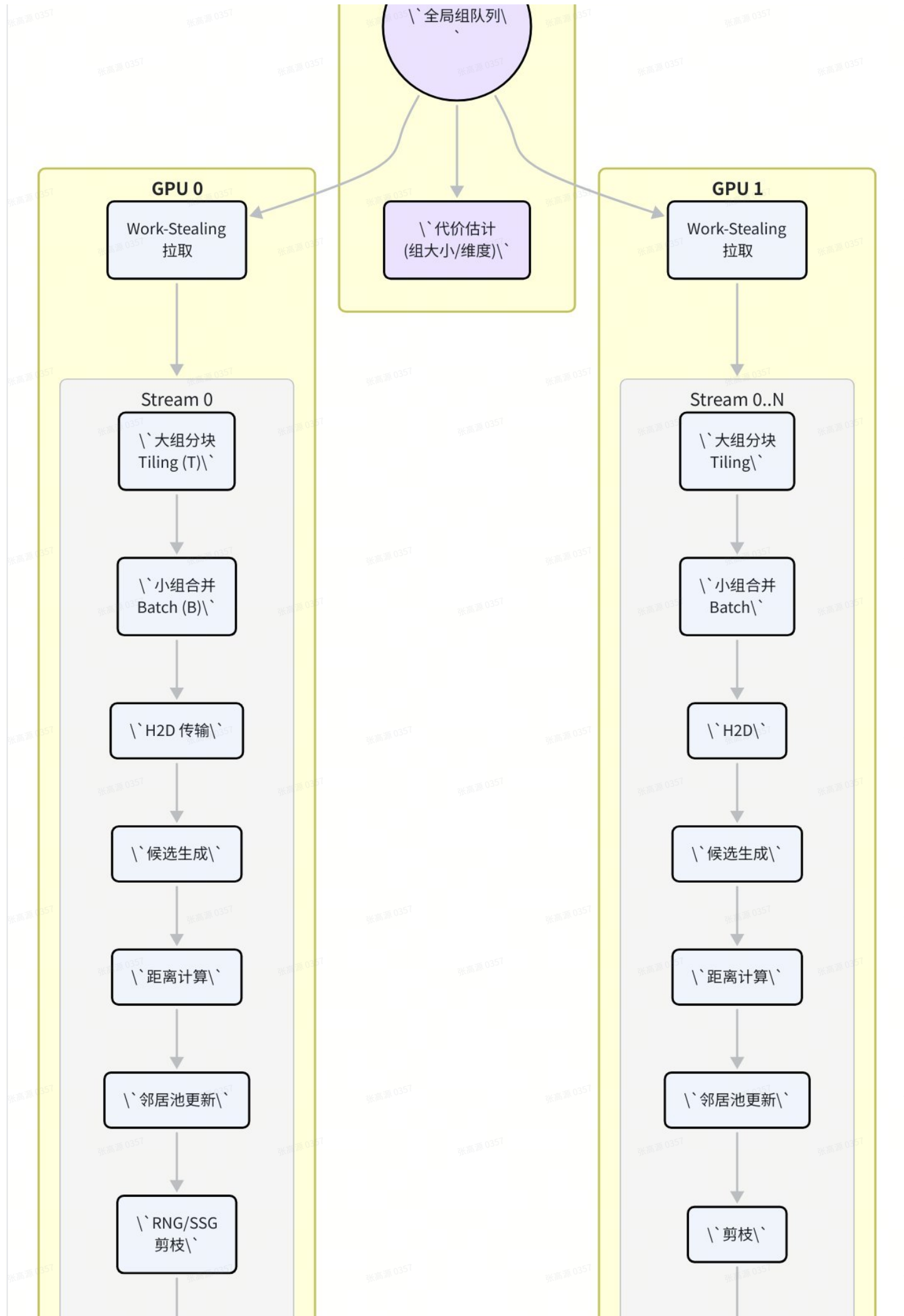
## • 系统级测试 (System-Level Testing):

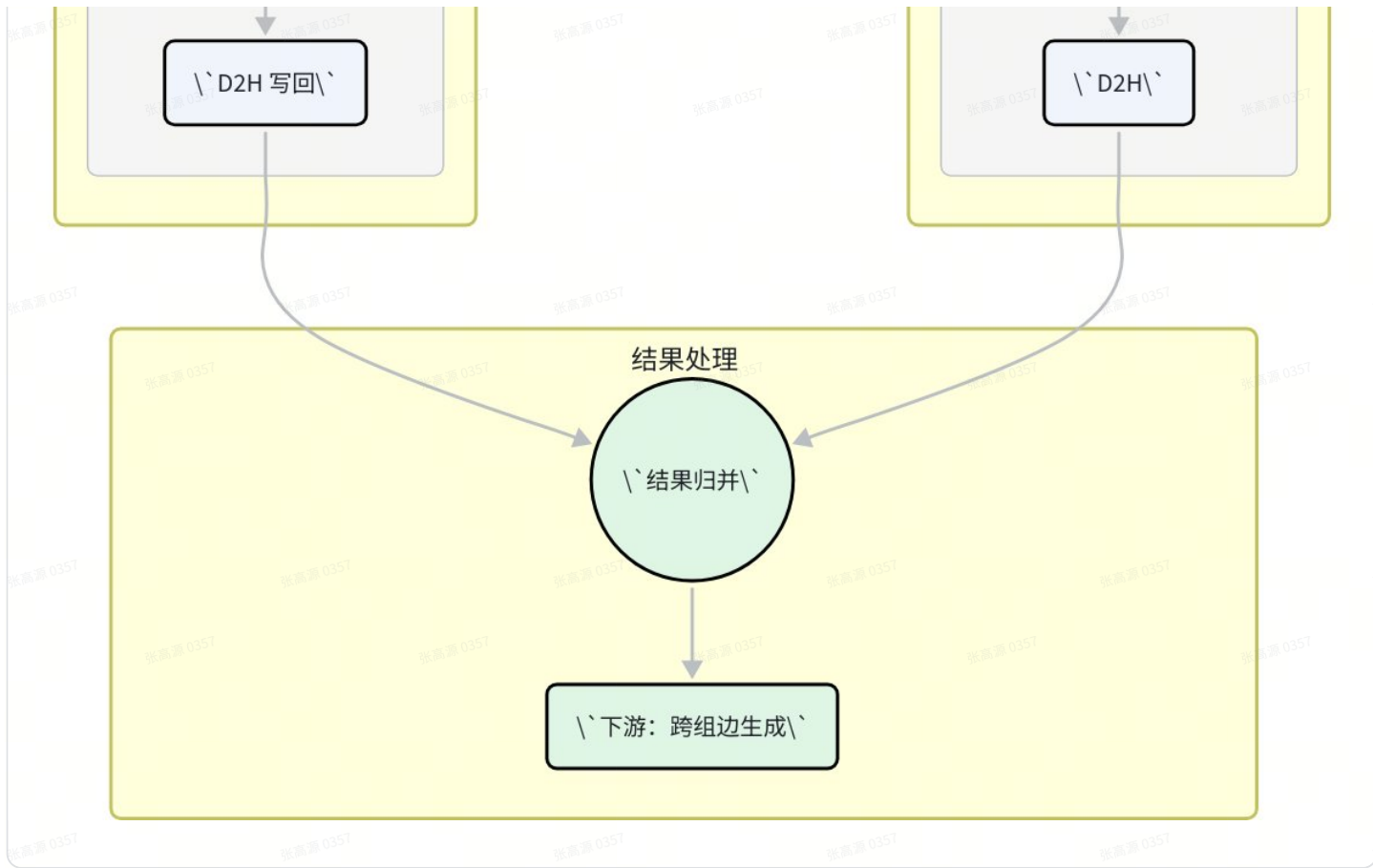
- **方法:** 运行完整的 `build_graph_for_all_groups` 流程, 并与 CPU 基线版本进行端到端对比。
- **评测脚本接口 ( `eval/eval_recall.py` ):**
  - 脚本应能通过命令行参数接收以下输入:
    - `--index_path` : 图文件路径。
    - `--query_path` : 查询向量文件路径。
    - `--groundtruth_path` : 真实近邻文件路径。
    - `--k` : 搜索的近邻数。
  - 脚本输出一个 JSON 对象, 包含 `recall@k`, `avg_latency_ms`, `qps` 等关键指标。
- **验收流程:**
  - 使用原始 CPU Vamana 代码, 在 SIFT1M 数据集上构建图, 记为 `cpu_graph.bin`。
  - 使用重构后的 GPU 代码, 在相同数据集和参数下构建图, 记为 `gpu_graph.bin`。
  - 分别运行 `eval_recall.py --index_path cpu_graph.bin ...` 和 `eval_recall.py --index_path gpu_graph.bin ...`。
  - 比较两个评测结果, 确保 GPU 版本的 `recall@k` 与 CPU 版本相当 (例如, 下降不超过 1%) , 同时构建时间显著缩短。

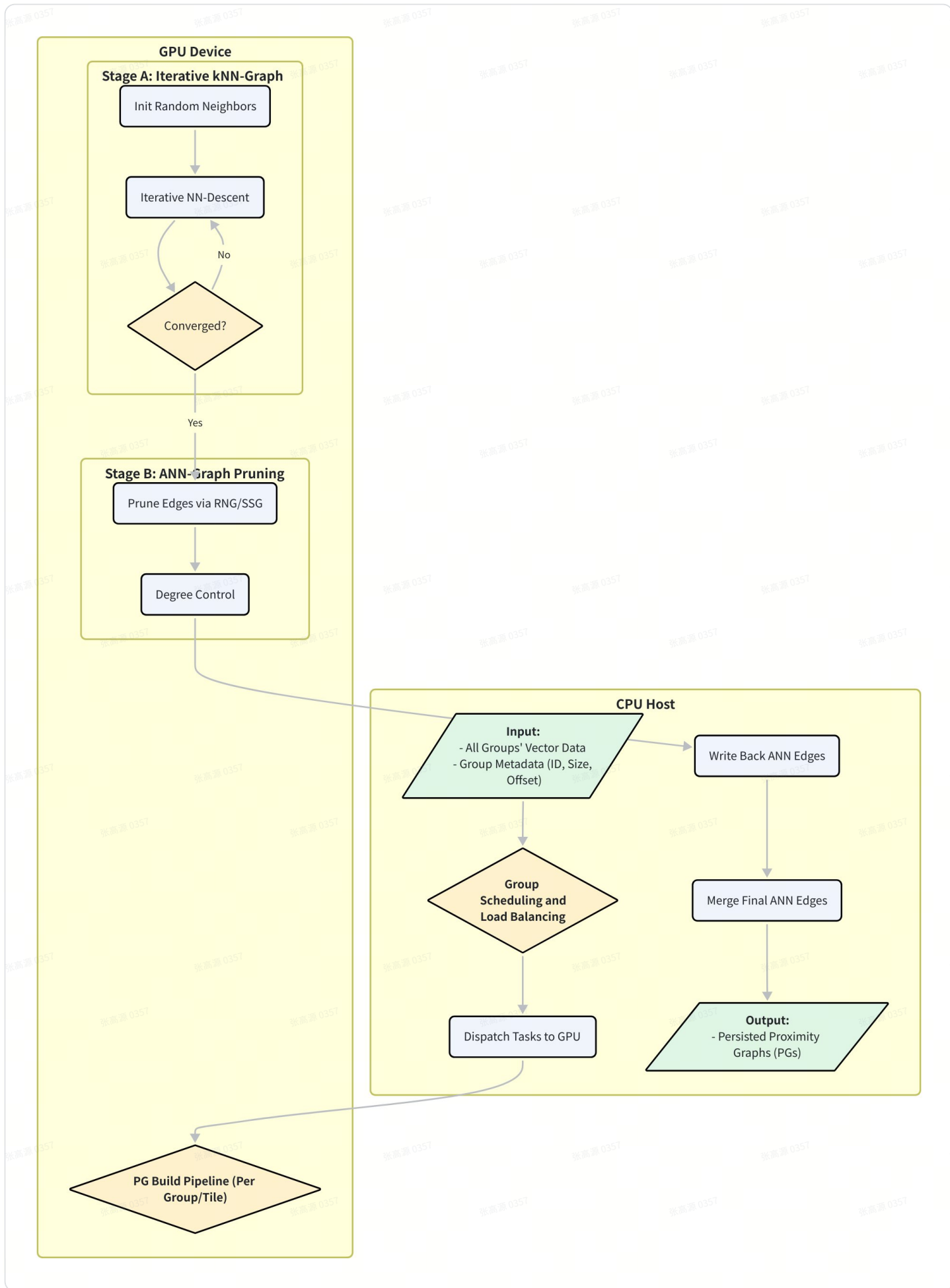
## 六、图示 (Diagrams)

### 6.1 端到端流程图









6.2 GPU 调度与负载均衡示意图

此图描绘了主机端调度器如何通过 Stream、分块（Tiling）、批处理（Batching）和工作窃取（Work-Stealing）等机制，将不同规模的构图任务高效地映射到 GPU 硬件上。

